pGPU-attention: Implement Attention Kernel in Pseudo-Assembly on a GPU SM Simulator

> "The true objective of software optimization is to reach the point where you must write assembly and command the raw bits of the hardware directly. Everything else is just a well-crafted excuse to get there." - Elias Bergström [1]

You rarely get to tinker close-to-metal on a real GPU — its hardware details are buried under layers of software, drivers, and virtualization abstractions, all designed to simplify the programmer's model while extracting maximum performance from the hardware. In this project, you'll work with a GPU simulator and write assembly code in its ISA to implement an attention kernel.

`pGPU` (pseudo-GPU) is a simulator for a minimal GPU-like device, written entirely in Python, with its own mini-ISA, DRAM, Scratchpad-Memory, Register-File and SM compute core models. The simulator comes with its own assembly language pPTX[2] (pseudo-PTX) that is used to program its ISA. The objective of this project is to implement an attention map computation kernel in this language and devise techniques to drive the runtime of the kernel down (latency cycles reported by the simulator) as close as possible to the theoretical minimum.

Problem Description

The attention map A, of *Attention Is All You Need*[3] fame, is computed on a query Q and a key K matrix as follows:

$$\text{A[b, h, :, :]} = \text{softmax}\left(\frac{Q[b, h, :, :] \times K[b, :, :]^\top}{\sqrt{d}}\right)$$

where,

Q is a matrix of shape (Batch, heads, n, d)

K is a matrix of shape (Batch, n, d), $K^\top$ (transpose along the (n, d) dims) is of shape (Batch, d, n)

A is a matrix of shape (Batch, heads, n, n)

and the *softmax* function is defined as:

$$\text{softmax}(\mathbf{z})_i = \frac{e^{\mathbf{z}_i - \max(\mathbf{z})}}{\sum_j e^{\mathbf{z}_j - \max(\mathbf{z})}}$$

The subtraction by $\max(\mathbf{z})$ in the above formula is performed to keep the computation numerically stable.

Your kernel implementation must cater to the following problem shape: $(\text{Batch} = 16, \text{heads} = 64, \text{n} = 8, \text{d} = 32)$[4]

Key Objectives

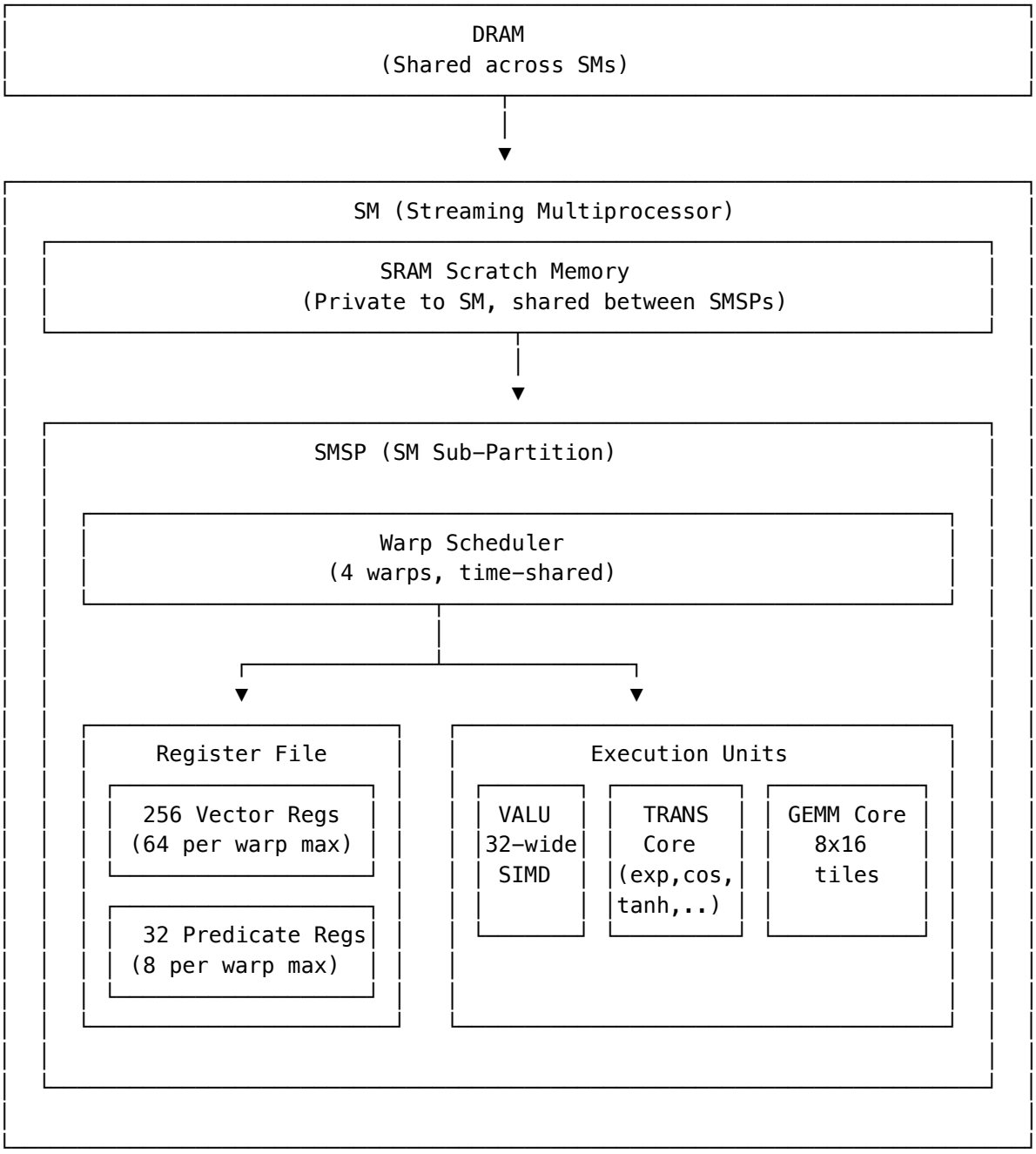
1. Map the operations required to perform the above computation to the ISA of the `pGPU` simulator, and write a kernel using the intrinsic functions and `asm` programming interfaces provided to you.
2. Use the `KernelTracer` and `objdump.py` tools in the simulator to analyze bottlenecks in your kernel and devise optimizations to circumvent them.
3. There are some gold nuggets / easter eggs embedded in the simulator for you to discover.
4. Map the learnings from working on this toy-ISA to the PTX / GCN ISA programming models of real GPUs!

Additional Reading

[5]If you found the problem statement interesting and want to know more details about the simulator and the project, please read the following section.

The pGPU Simulator

pGPU is a python-based single-SM simulator modelling various aspects of a GPU's SM.



Notes: - The baseline simulator only supports 1 SMSP per SM - Each SMSP's 256 vector registers and 32 predicate registers are statically partitioned between 4 warps - Per-warp register limits: 64 vector registers, 8 predicate registers

The simulator uses a naive round-robin scheduling algorithm to switch between warps in an SMSP when a warp is stalled on a long latency instruction.

Software Ecosystem and Tooling

- pGPU exposes an assembly like interface for programming the simulated ISA. Check out the `assemble_instruction` function in pgpu/sw/kernel.py for the asm syntax and the `WarpKernel.asm()` function in the same file for inserting inline assembly into your kernel. An example vector-add kernel is provided in pgpu/examples/vector_add.py.
- It is recommended to use and develop intrinsic functions (or rather pre-defined inline-asm blocks) to wrap commonly used pPTX asm routines in your kernel. This avoids clutter in your kernel and simplifies your optimization workflow. You can find some in-built intrinsic functions in pgpu/sw/intrinsics.py.
- The simulator comes with an `objdump.py` python script and a perfetto-compatible `KernelTrace` tool.
  - Use `objdump.py` to dump out the pPTX assembly that your kernel corresponds to — this will come in handy when you have a bunch of intrinsic calls in your kernel. `objdump.py` also allows you to print the set of live registers for each pPTX asm instruction in your kernel (mirroring an interface similar to `nvdisasm -lrm`).
  - `KernelTrace` traces the simulation of your kernel and generates a json file that you can open with perfetto to see a timeline-view of the instructions executed by each warp (NVIDIA's `nsight` tools do not give you that! `pGPU-1 NVIDIA-0`).
- Finally, the entire source-code of the simulator is made available to you. You are encouraged to tinker with the pgpu/sw and pgpu/devtools folders to add tools to simplify your workflow.

Pre-empted FAQs

1. Where it differs from NVIDIA's PTX model

1. The architectural models and the latencies of the hardware resources in pGPU may be different from those on a real GPU. This means the trade-offs you prioritize when writing a kernel in pPTX may be different from the wisdom you have accumulated for an NVIDIA GPU in the GPU Programming Course.
2. pGPU is much simpler than a real GPU — given the fact that the simulator only has a single SM Sub-Partition and that it uses a naive warp scheduling algorithm. This will simplify the programming experience when writing pPTX while also planting inefficiencies in your kernel that may be difficult to circumvent.
3. pGPU lacks most of the virtualization layers you would see in a GPU allowing you to play with the low-level hardware model (albeit simulated) to a better extent.
4. pGPU ISA is different from PTX ISA with the differences being substantial at some places. Some of these differences are detrimental to performance, while some may be to your advantage.

2. Where it is NVIDIA-like

1. Your key objectives do not change: prioritize DRAM memory coalescing, avoid shared memory bank conflicts, improve instruction-level parallelism within a warp and avoid redundant data movement when writing performant kernels.
2. The pGPU hardware model is built to mimic the actual hardware model of an NVIDIA GPU. While it is not an exact replica of an NVIDIA GPU's SM, it is a good enough approximation around which you can build a mental model for performance optimization.[6]

3. Finally, the programming interface is made to look somewhat like an actual CUDA / PTX kernel. This is just there to help with writer's block one may run into when writing kernels in assembly.

3. How should I approach this problem?

1. Read the docs and go through the files in the pgpu/sw folder to get a hold of the simulator features and the software APIs around it.
2. Read the ISA! You might find unconventional instructions here that might come in handy for your kernel.
3. Repeat the traditional kernel performance optimization loop: Build Hypotheses for Performance Bottlenecks $\rightarrow$ Implement Optimization $\rightarrow$ Profile and Inspect ASM $\rightarrow$ Validate Hypothesis. The simulator workflow here allows you to iterate over this loop faster than in the real-world.
4. Bombard your mentors with ideas and enthusiasm in your regular meetings with them during the project!

4. Why program in pPTX when I can program in PTX on real GPUs? ***(aka Why should I pick this problem: a canvassing attempt from the author)***

1. Per FAQ answers 1.3 and 2.2, this problem gives you visibility to some close-to-metal traits of an actual GPU that are usually buried behind the multitude of GPU software, driver and virtual-memory abstractions that come in your way. Things you do not get to see on real GPUs that easily.
2. This problem is devised with the intention of leading you towards developing a better understanding of a real GPU with some gold-nuggets planted along the way. We believe these nuggets will help you discover the nuances in a GPU and appreciate its architecture even more than you currently do.
3. *So you think you can write a speed-of-light attention kernel in PTX? Try it on a simpler (simulated) GPU first!*
4. Have a packed sem and don't know if you have the bandwidth to take this up? Been there, done that. You can reach out to me if you want help deciding: `arjunmenon.vadakkeveedu@kla.com`.

5. Using LLMs to Code in pPTX [7]The following points are gated by the official AI usage and honor code for the CS6023 course. If LLM tools are strictly forbidden in the course, you needn't even read the following points.

1. Please follow honor code and don't let LLMs rob you of the joy of performance optimization and discovering the gold nuggets in this problem!
2. Since the simulator is fully open-source, please avoid getting it indexed by a coding agent.
3. If some amount of LLM usage is allowed in the course, you may use LLM chatbots to clarify doubts, iterate over ideas and explain code snippets. But please refrain from running a prompt like `/goal Modify the following kernel until it meets the theoretical minimum runtime for the simulator model.`

Relevant Links

- pGPU Simulator: arjunmenonv/pGPU
- docs and example kernels: docs, vector-add, intrinsic-demo
- Read the code!

  "Always read the code if you have access to it — programs and commit histories tell you stories that volumes of documentation forget to capture." - yet another quote by Elias Bergström

[1] No such person or quote exists to my knowledge, engineers are pragmatic people after all. But if you find programming in GPU assembly exciting and resonated with the quote when you first read it, this project might be for you. ↩

[2] Just like the popular commercial software with the same name, this language can also hang indefinitely if you aren't careful with parallel writes. But at least things are under your control here. ↩

[3] Sneaking in a reference to Vaswani et al. here for engagement, good luck and prosperity. ↩

[4] Not happy seeing that the attention map is only operating over 8 tokens? We need your help to scale the kernel to support a larger context length! ↩

[5] Ran into the 1-page limit set for the problem statement! You aren't supposed to read what comes after this point (*but really, READ it*) ↩

[6] One could say it is built atop the mental model I have built for myself over a few years of performance optimization on GPUs, and is likely to overlook many details and have many simplifying assumptions. ↩

[7] The intention behind this problem was to cut the middle ~~man~~ LLM out from GPU kernel optimization. These tools have become quite effective over the past few months at writing CUDA and have robbed many of us of the creative joy of implementing and validating kernel optimizations. Surely no frontier LLM was trained on writing pPTX assembly, after all it was something I made up! But given that LLMs now are capable of solving Millennium Problems and the disheartening fact that I am no Navier, Stokes, or Riemann, I stand no chance against these tools. ↩